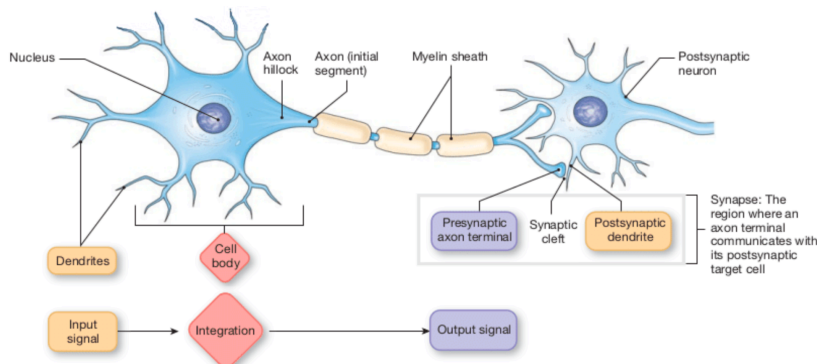


Deep Learning

Deep learning is a part of machine learning where computers learn from data using structures called artificial neural networks. These networks are inspired by how the human brain works. The main goal is to help machines recognize patterns and make decisions, like humans do.



Basic Structure

A deep learning model is made of layers:

- Input layer: Takes the raw data (like images, text, or numbers)
- Hidden layers: Do most of the learning and processing
- Output layer: Gives the final result (prediction or classification)

Each layer is made of small units called neurons. A neuron:

1. Takes input values
2. Multiplies them with weights
3. Adds them together
4. Passes the result through an activation function (like ReLU or Sigmoid)

Activation functions help the network learn non-linear patterns, meaning complex relationships in data.

How Learning Happens

Deep learning works in two main steps:

1. Forward Propagation

Data moves from input → hidden layers → output.

The model makes a prediction based on what it has learned.

2. Backpropagation

The model checks how wrong its prediction is using a loss function.

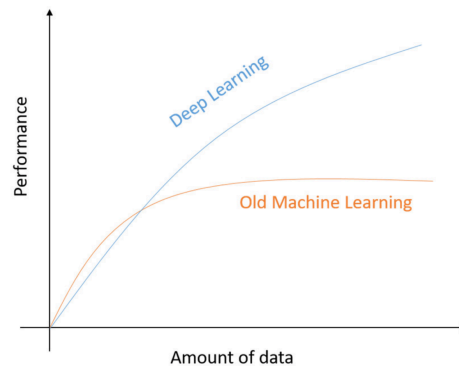
Then it sends this error backward and adjusts the weights using gradient descent.

This helps the model improve over time.

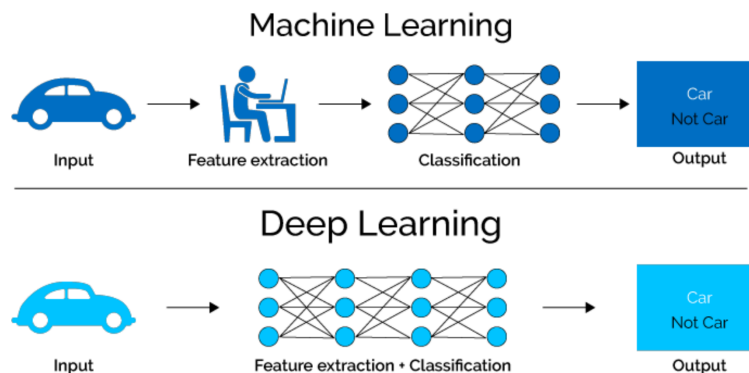
Why Deep Learning is Useful

Deep learning is powerful because:

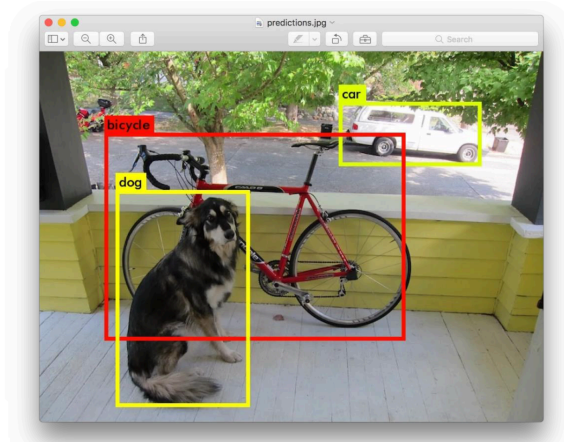
- It works very well with large amounts of data



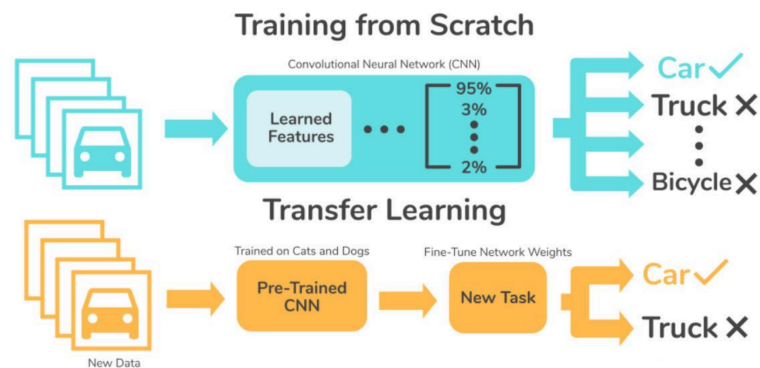
- It automatically learns important features (no need to manually design them)



- It can handle complex data like images, speech, and text



- Can reuse knowledge via transfer learning



Common Types of Deep Learning Models

- MLP (basic neural network): Works on simple structured data
- CNN: Used for images (face recognition, medical imaging)
- RNN: Used for sequences like text or time series
- Transformers: Used in modern AI like ChatGPT
- GANs: Used to generate fake but realistic data (images, etc.)

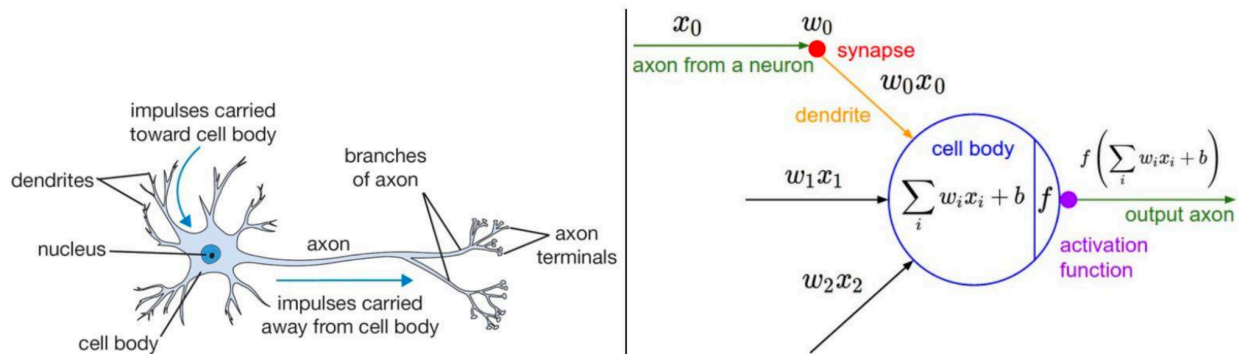
Challenges

- Needs a lot of data and computing power
- Hard to understand how decisions are made (“black box”)
- Can sometimes memorize data instead of learning general patterns (overfitting)

- Training can be unstable due to gradient issues

Artificial Neurons Motivation

Artificial neural networks are inspired by the structure of the human brain. In the brain, neurons communicate with each other to process information. Artificial neurons try to copy this idea in a simplified mathematical way.



Dendrites (Input Side)

In the brain, dendrites receive signals from other neurons.

In artificial neurons, this is like the input values coming into the model.

Cell Body (Soma) → The Main Processor

The cell body collects all incoming signals and decides what to do.

In an artificial neuron:

- Inputs are multiplied by weights
- Then added together
- Then passed through an activation function

This is where the main “decision-making” happens.

Nucleus → Decision Logic (Idea)

The nucleus controls the cell’s behavior.

In artificial neurons, this is similar to:

- weights and bias
They control how important each input is and guide the final output.

Axon (Output Side)

The axon sends the processed signal to other neurons.

In neural networks:

- The output of one neuron becomes input for the next layer

Synapse (Connection Strength)

A synapse connects two neurons in the brain, and its strength can change over time.

In artificial neural networks:

- This is represented by weights
- Strong weight = strong influence
- Weak weight = less influence

During training, these weights are adjusted so the network learns better.

Human Brain → Neural Networks

Artificial neural networks are inspired not only by the structure of brain neurons but also by how the human brain processes information and learns.

1. Layered Learning (Hierarchical Processing)

The human brain does not understand everything at once. It learns step by step:

- First, it detects simple features (like edges in an image)
- Then it combines them into shapes
- Finally, it recognizes full objects (like a face or a car)

Neural networks do the same:

- First layers learn simple patterns
- Middle layers learn more complex features
- Final layers make decisions

This is called hierarchical learning.

2. Activation Functions (Non-linear Thinking)

Brain neurons do not just switch ON or OFF. Their response depends on how strong the input signal is.

Similarly, artificial neurons use activation functions (like ReLU or Sigmoid) to decide:

- whether to activate strongly
- weakly
- or not at all

This helps the model learn complex, non-linear patterns, not just simple straight-line relationships.

3. Learning from Experience

The brain learns by experience:

- Frequently used connections become stronger
- Rarely used ones become weaker

Neural networks learn in a similar way:

- They are trained using data
- They adjust weights based on errors
- This is done using an algorithm called backpropagation

So, the model improves gradually, just like humans learning from practice.

4. Parallel Processing

The brain can process many signals at the same time.

Neural networks also:

- Process many inputs simultaneously
- Each neuron works in parallel

This makes them very fast and powerful for tasks like:

- image recognition
- speech recognition
- language processing

Neural Networks

Neural Networks are computer models inspired by how the human brain works. They are widely used in machine learning and artificial intelligence to find patterns in data and make decisions.

At a basic level, a neural network is made of many small units called neurons (or perceptrons) that are connected together in layers.

The main idea of NN is the network learns from data, processes it step by step, and produces an output like a prediction or classification.

Structure of a Neural Network

A neural network has three main layers:

1. Input Layer

- This is the first layer
- It receives raw data (called features)
- Example: pixels of an image, numbers in a dataset, or words in a sentence

2. Hidden Layers

- These are the middle layers

- They process and transform the data using mathematical operations
- Each layer learns patterns from the previous layer

If there are many hidden layers, it is called a Deep Neural Network

3. Output Layer

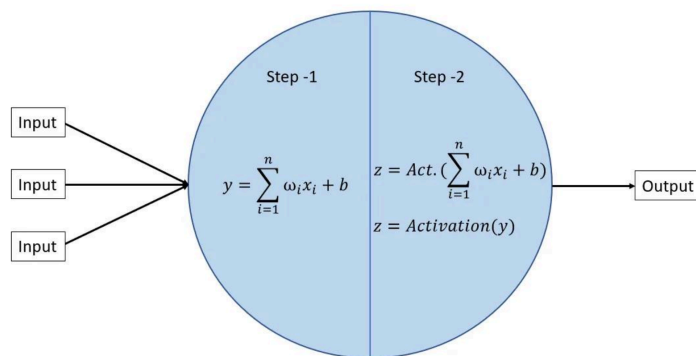
- This is the final layer
- It gives the result of the network
- Example: “cat or dog”, price prediction, sentiment (positive/negative)

Neuron/Unit Structure

A neuron (or unit) in a neural network is the basic building block that processes information. It is inspired by a biological neuron in the brain and helps the network learn from data.

What a Neuron Does

A neuron takes input data, processes it step by step, and produces an output.



Step-by-step process:

1. **Input**
 - The neuron receives input values (data/features).
2. **Weights**
 - Each input is multiplied by a weight.
 - Weights decide how important each input is.

3. Weighted Sum

- All weighted inputs are added together.
- A bias is often added to adjust the result.

4. Activation Function

- The result is passed through an activation function (like ReLU or Sigmoid).
- This adds non-linearity, helping the model learn complex patterns.

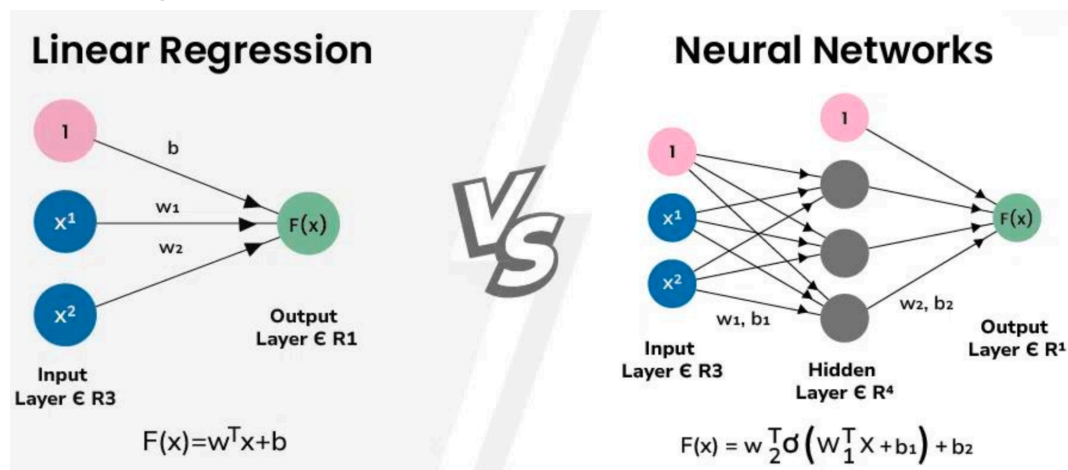
5. Output

- The final value is sent to the next layer or used as the prediction.

Facts about Neurons and Neural Networks:

- Neurons are arranged in layers and layers are arranged sequentially.
- Neurons within the same layer do not transfer information with each other.
- All data/info entered via the input layer and information/output goes out by the output layer.
- Neurons in layer “L” are connected to neurons in layer “L+1”.
- Every interconnection in the neural network has a weight associated with it, and every neuron has a bias associated with it.
- All neurons in a particular layer use the same activation function.

Linear Regression vs Neural Networks



Linear regression can be seen as a single-layer neural network because it has the same basic structure as one neuron:

- Inputs are directly connected to the output (no hidden layers)

- Each input is multiplied by a weight and summed with a bias
- The output is just this weighted sum (no non-linear activation)

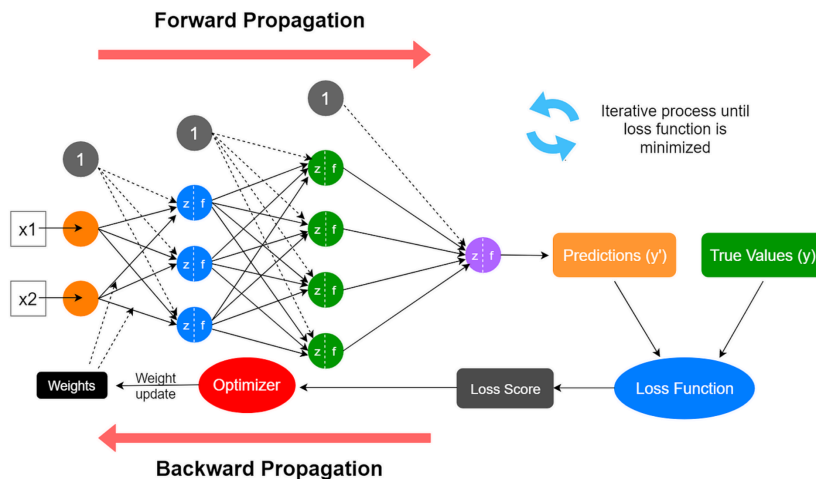
So mathematically:

$$F(x) = w^T x + b$$

This is exactly what a single neuron does without an activation function, which is why linear regression is considered a single-layer neural network.

Neural Networks

A neural network learns through a repeated cycle of prediction and correction.



1. Forward Propagation

- Input data ($x_1, x_2, \dots$) enters the network
- It passes through hidden layers
- Each neuron computes weighted sum + activation
- The network produces a prediction ($\hat{y}$)

2. Loss Calculation

- The prediction ($\hat{y}$) is compared with the real value (y)
- A **loss function** measures the error (how wrong the prediction is)

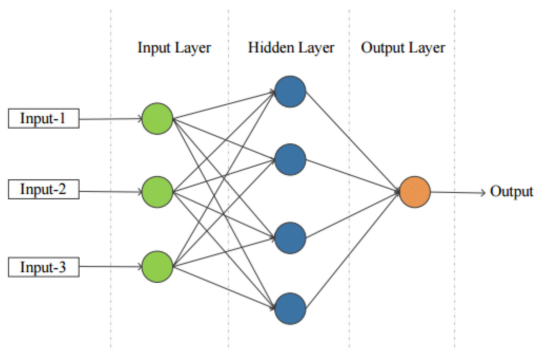
3. Backpropagation + Optimization

- The error is sent backward through the network
- The model finds which weights caused the error
- An optimizer (like Gradient Descent) updates the weights

4. Repetition (Learning Loop)

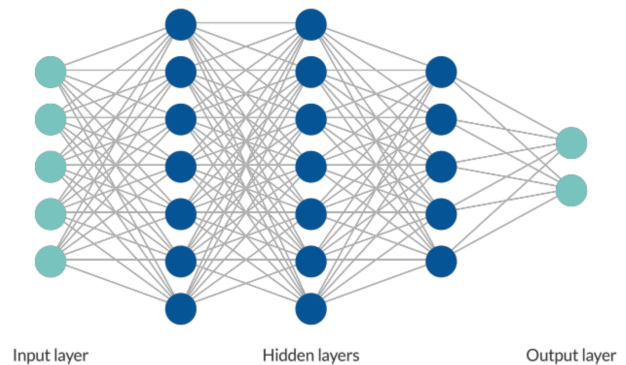
- The process repeats many times
- Each cycle reduces the error
- Eventually, the model becomes more accurate

SHALLOW NEURAL NETWORKS



A Feedforward Neural Network with One Hidden Layer

DEEP NEURAL NETWORKS



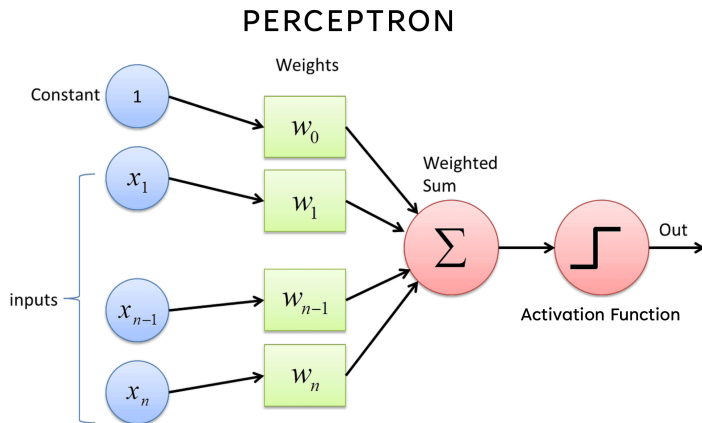
A Deep Neural Network with Three Hidden Layers

Perceptron

A perceptron is the simplest form of a neural network. It represents a single artificial neuron and is the basic building block of all deep learning models.

A perceptron:

takes inputs → weights them → sums them → applies activation → produces output



How a Perceptron Works

1. Inputs and Weights

- It receives input values (features)
- Each input is multiplied by a **weight** (importance factor)

2. Weighted Sum (Core Calculation)

- All weighted inputs are added together
- A bias is added to adjust the result

This step is similar to linear regression:

$$z = w^T x + b$$

$$f_w(x) = \sum_{i=0}^n w_i x_i$$

3. Activation Function

- The result is passed through an activation function (ReLU, Sigmoid, etc.)
- This introduces non-linearity

4. Output

- The perceptron produces a final output
- This can be a prediction or input to the next layer

Limitation

- A single perceptron can only solve simple linear problems
- It cannot learn complex patterns on its own

Multi-Layer Perceptron (MLP)

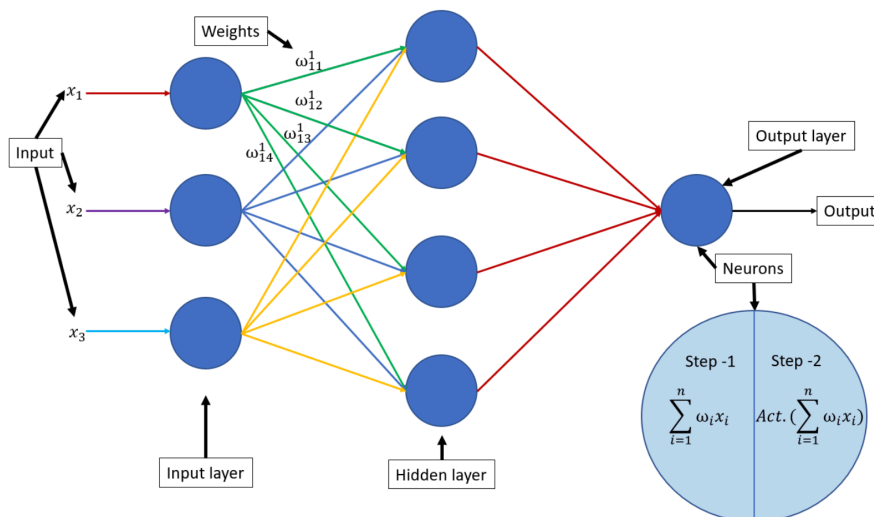
A Multi-Layer Perceptron (MLP) is a basic type of neural network where many neurons are arranged in multiple layers. It is an extension of a single perceptron (single neuron) and is also called a feedforward neural network.

MLP is feedforward, meaning:

Input $\rightarrow$ Hidden Layers $\rightarrow$ Output (one direction only)

No backward flow during prediction.

Structure of an MLP



An MLP has three main parts:

1. Input Layer

- Takes raw data (features)
- Example: pixels, numbers, text features

2. Hidden Layer(s)

- One or more layers between input and output
- Each layer transforms the data step by step
- Learns complex patterns from the input

This is what makes MLP more powerful than a single perceptron.

3. Output Layer

- Produces the final prediction
- Example: class label, probability, or numeric value

Key Characteristics

1. Fully Connected Network

- Every neuron connects to every neuron in the next layer
- Each connection has a weight

2. Non-Linearity

- Hidden layers use activation functions (ReLU, etc.)
- This allows learning complex, non-linear patterns

How MLP Learns

1. Forward Propagation

- Data passes through layers

- Model makes a prediction

2. Loss Calculation

- Prediction is compared with actual value
- Error is measured using a loss function
 - MSE → regression
 - Cross-entropy → classification

3. Backpropagation + Optimization

- Error is sent backward
- Weights are updated using **gradient descent**
- This reduces future errors

Forward Propagation

Forward propagation is the process where input data moves through a neural network to produce a prediction.

Forward propagation means: Input data flows forward → gets transformed layer by layer → produces output

Step-by-step Process

1. Input

- Raw data is given to the network
- Example: numbers, pixels, or features

2. Weighted Sum (Transformation)

- Each neuron receives inputs
- Inputs are multiplied by weights
- A bias is added

$$z = w^T x + b$$

3. Activation Function

- The result is passed through an activation function
- Examples: ReLU, Sigmoid, Tanh, Softmax
- This adds non-linearity, helping the model learn complex patterns

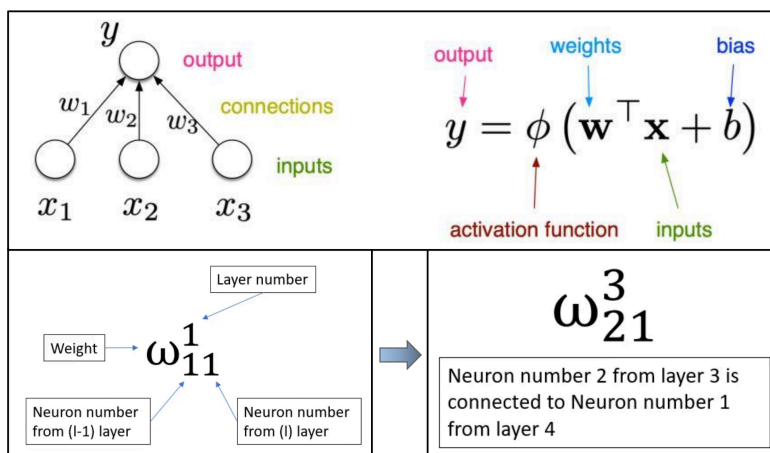
4. Passing Through Hidden Layers

- The output of one layer becomes input to the next layer
- This continues layer by layer
- Each layer learns more abstract features

5. Output

- Final layer produces the prediction
- This could be:
 - a class (classification)
 - a number (regression)
 - Probabilities

FORWARD PROPAGATION TERMINOLOGIES

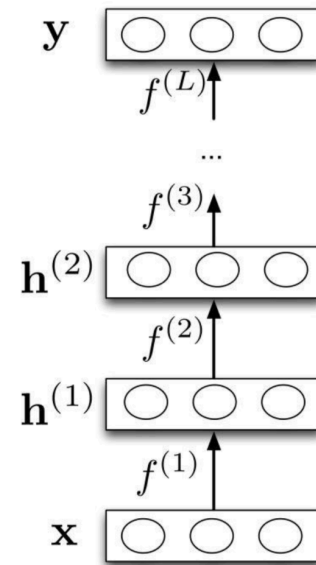


Let us assume that we have an L layer Neural Network ($L = L-1$ Hidden Layers + 1 Output Layer). $\mathbf{h}^{(i)}$ is the vector of the values of neurons (called **activations/outputs**) in the i^{th} hidden layer. The function $\mathbf{f}^{(i)}$ represents the overall transformation from $(i-1)^{\text{th}}$ layer to i^{th} layer. Finally, $\phi(\mathbf{z})$ represents the **activation function**.

Forward Propagation Through the Layers

Each layer computes a function in the neural network, so, overall, the network computes a **composition of functions**.

$$\begin{aligned}\mathbf{h}^{(1)} &= \mathbf{f}^{(1)}(\mathbf{x}) = \phi(\mathbf{w}^{(1)\text{T}}\mathbf{x} + \mathbf{b}^{(1)}) \\ \mathbf{h}^{(2)} &= \mathbf{f}^{(2)}(\mathbf{h}^{(1)}) = \phi(\mathbf{w}^{(2)\text{T}}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}) \\ &\vdots \\ \mathbf{h}^{(i)} &= \mathbf{f}^{(i)}(\mathbf{h}^{(i-1)}) = \phi(\mathbf{w}^{(i)\text{T}}\mathbf{h}^{(i-1)} + \mathbf{b}^{(i)}) \\ &\vdots \\ \mathbf{y} &= \mathbf{f}^{(L)}(\mathbf{h}^{(L-1)})\end{aligned}$$



Forward Propagation Through the Layers

So, Overall, we can represent the input to output mapping as:

$$\mathbf{y} = \mathbf{f}^{(L)} \cdot \mathbf{f}^{(L-1)} \cdot \dots \cdot \mathbf{f}^{(2)} \cdot \mathbf{f}^{(1)}(\mathbf{x})$$

In the **output layer**,

- If the task is **Regression**:

$$\mathbf{y} = \mathbf{f}^{(L)}(\mathbf{h}^{(L-1)}) = \mathbf{w}^{(L)\text{T}}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}$$

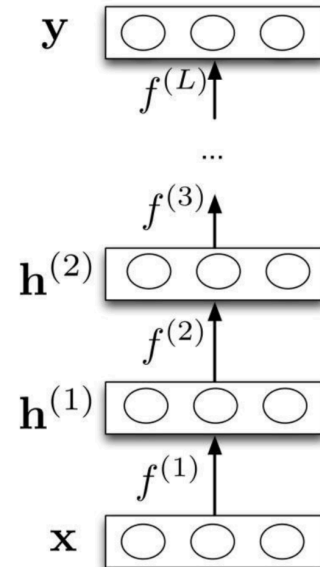
- If the task is **Binary Classification**:

$$\mathbf{y} = \mathbf{f}^{(L)}(\mathbf{h}^{(L-1)}) = \sigma(\mathbf{w}^{(L)\text{T}}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)})$$

- If the task is **Multinomial Classification** :

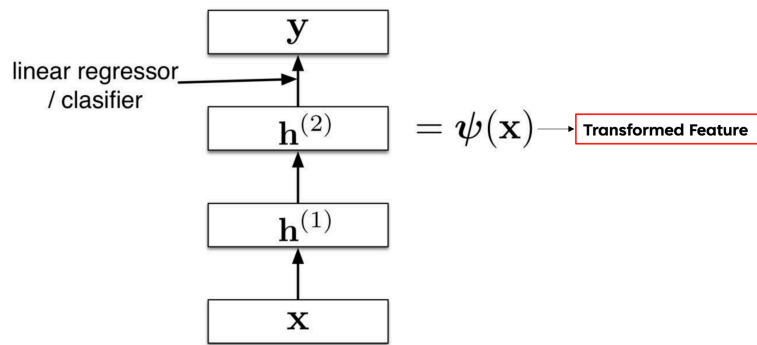
$$\mathbf{y} = \mathbf{f}^{(L)}(\mathbf{h}^{(L-1)}) = \text{SoftMax}(\mathbf{w}^{(L)\text{T}}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)})$$

Finally, if it's a classification task, then **thresholding** or **argmax** is applied to generate the predicted label.

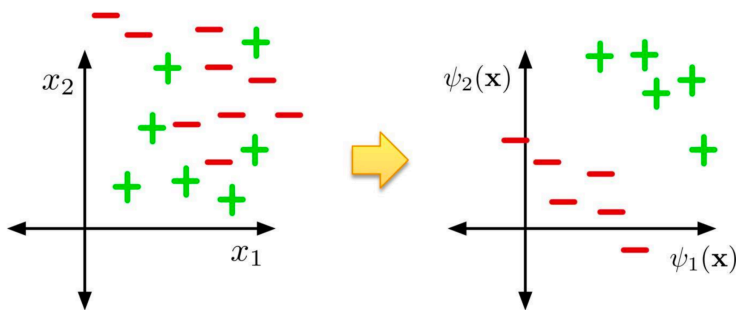


Feature Learning

Feature learning is one of the most powerful ideas in deep learning. It means the neural network can automatically discover important patterns from raw data without humans manually designing features.



The Goal: Separation of Classes in the Transformed Feature Space



Traditional Machine Learning vs Deep Learning

Traditional ML

- Humans must manually choose features
- Example: edges, textures, word counts, etc.
- This requires domain knowledge and effort

Deep Learning

- The model learns features by itself
- Works directly on raw data like images, text, audio

How Feature Learning Happens

Feature learning happens during forward propagation:

- Data enters the network

- Each hidden layer transforms it using:
 - weighted sums
 - activation functions

$$z = w^T x + b$$

- Each layer produces a new representation of the data

From Simple to Complex Features

As data moves deeper into the network:

- Early layers → detect simple features (edges, colors, textures)
- Middle layers → detect shapes and patterns
- Deep layers → detect full objects (faces, cars, dogs)

So the network builds understanding step by step.

Goal of Feature Learning

The main goal is:

Transform raw data into a space where classes become easily separable.

- In raw form → data is messy and overlapping
- After transformation → classes become well separated
- Final layer can easily make predictions

Why It Is Powerful

- No manual feature engineering needed
- Adapts automatically to data
- Works well with images, text, and audio

Limitation

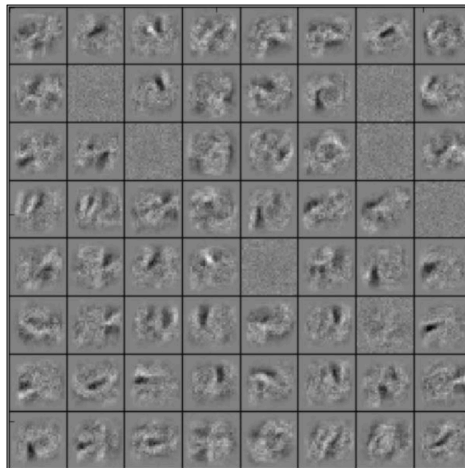
Because features are learned automatically:

- Humans cannot easily interpret them
- This creates the “black box” problem

We know it works well, but not always *exactly why*.

FEATURE LEARNING

Here are some of the features learned by the first hidden layer of a handwritten digit classifier (after rearranging the **activations** as images):



Unlike hard-coded feature maps (e.g., in polynomial regression), features learned by neural networks **adapt to patterns** in the data.

Activation Functions

An activation function is a rule inside a neuron that decides whether the neuron should “activate” and how strong its output should be.

Its main job is to introduce non-linearity, because without it, a neural network would behave like a simple linear model no matter how many layers it has.

It can also scale outputs into useful ranges like probabilities.

Common Activation Functions

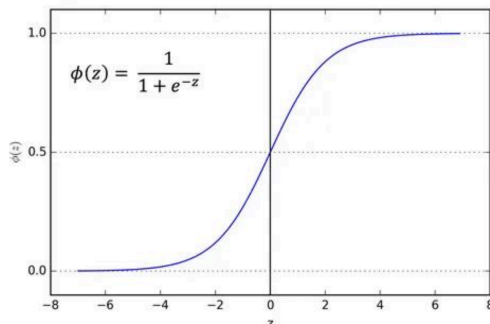
1. Identity (Linear)

- Output = input
 $f(z)=z$
- Used in regression output layer
- No non-linearity $\rightarrow$ not used in hidden layers

2. Sigmoid

- Converts input into range 0 to 1

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

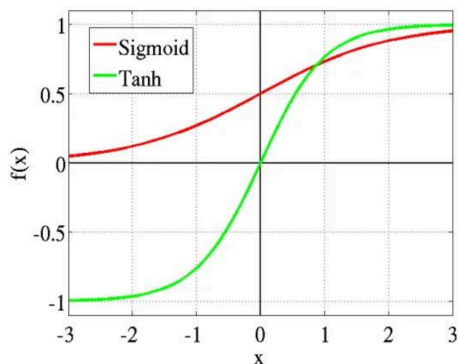


- Used for binary classification (probabilities)
- Problem: vanishing gradient (learns slowly in deep networks)

3. Tanh

- Output range: **-1 to 1**

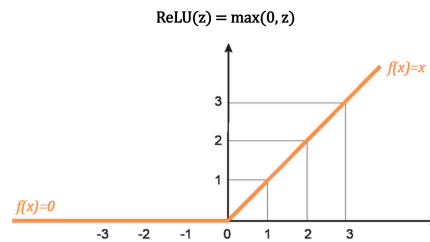
$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



- Better than sigmoid for hidden layers
- centered outputs (zero-centered)
- Still suffers from vanishing gradients

4. ReLU (Most common)

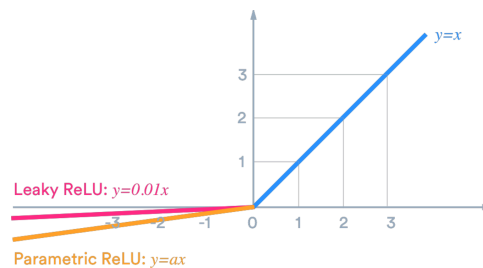
- Outputs:
 - $\max(0, \text{input})$



- Fast and widely used in hidden layers
- Problem: “dying ReLU” (some neurons stop learning)

5. Leaky ReLU

- Fixes dying ReLU by allowing small negative output



- Keeps neurons alive even for negative inputs
- Common improvement over ReLU

6. Softmax

- Converts outputs into probabilities that sum to 1

$$\text{SoftMax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} = \frac{\exp(z_i)}{\exp(z_1) + \exp(z_2) + \dots + \exp(z_K)}$$

Where:

- z_i is the raw output (logit) of class i ,
- K is the number of classes,
- The numerator is the exponent of the raw output for class i ,
- The denominator is the sum of the exponentials of the logits for all classes.

- Used in multi-class classification (final layer)

- Example: cat vs dog vs horse

Generalization in Neural Networks

Generalization means how well a neural network performs on new, unseen data, not just the data it was trained on.

A good model should not only “memorize” training data, but also understand patterns that work in real life.

Overfitting

Deep neural networks have:

- Many layers
- Many neurons
- A large number of parameters (weights + biases)

This gives them very high learning power, but also a risk:

They may memorize training data instead of learning patterns

This is called overfitting.

What overfitting means:

- Works very well on training data
- Performs poorly on new data

Why Neural Networks Overfit Easily

- They are very powerful models
- They can fit even noise and random patterns
- A single hidden layer network can already approximate very complex functions if large enough

So, without control, they may “learn too much detail” instead of general rules.

Goal: Good Generalization

We want the model to: Learn real patterns, not memorized examples

So it performs well on both:

- Training data
- Test/unseen data

How We Improve Generalization (Regularization)

1. L2 Regularization

- Penalizes very large weights
- Encourages simpler models

2. Data Augmentation

- Creates modified versions of training data
- Example: rotated or flipped images
- Helps model see more variety

3. Early Stopping

- Stop training when performance on validation data starts getting worse
- Prevents over-learning

4. Dropout

- Randomly “turns off” some neurons during training
- Forces the network to not depend on specific neurons
- Makes learning more robust

Dropout

Dropout is a technique used in neural networks to reduce overfitting and improve generalization.

Main Idea

During training, dropout: randomly “turns off” some neurons so the network does not rely too much on any single neuron.

This forces the model to learn stronger and more general features.

How Dropout Works

During Training:

- Each neuron is randomly deactivated with a probability p (dropout rate)
- Example: if $p = 0.5$
→ about 50% neurons are turned off each step

So:

- Only a **random subset of neurons is active** each iteration
- The network behaves like many smaller networks working together

Example

If a layer has 100 neurons and dropout = 0.5:

- ~50 neurons are active per training step
- The rest are temporarily ignored

Why This Helps

Dropout:

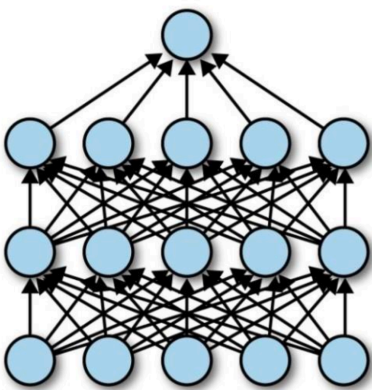
- Prevents the model from relying too much on specific neurons

- Forces redundancy (multiple ways to learn the same pattern)
- Reduces overfitting
- Improves robustness

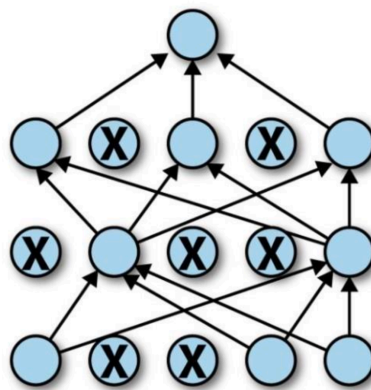
During Testing

- No neurons are dropped
- All neurons are used
- Outputs are scaled to match training behavior

This ensures consistent predictions between training and testing.



Without Dropout



With Dropout

Batch Normalization

Batch Normalization is a technique used in deep neural networks to make training faster, more stable, and more reliable.

It works by keeping the values inside the network in a consistent range so learning does not become unstable.

Why It Is Needed

During training:

- Outputs of layers can become very large or very small

- This causes unstable gradients
- Learning becomes slow or difficult

Batch Normalization fixes this.

How Batch Normalization Works

It is applied to activations of a layer in three steps:

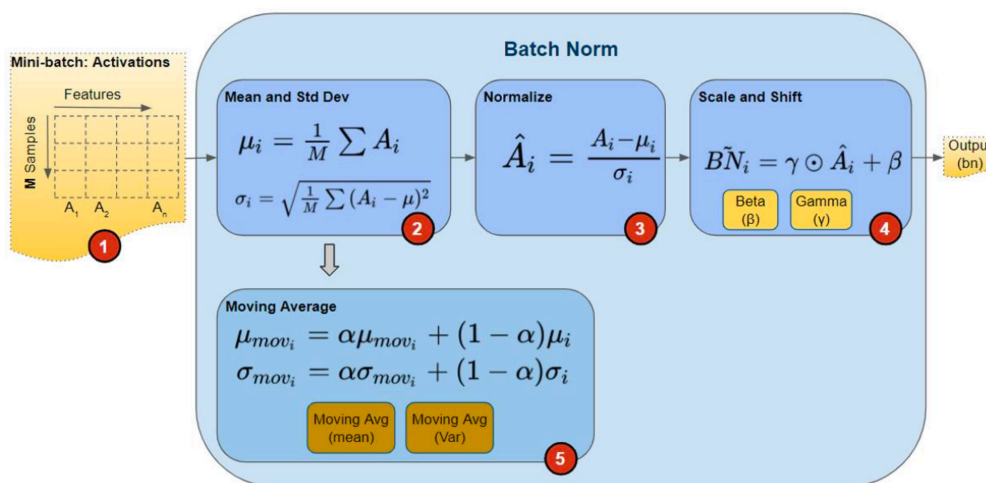
1. Compute Mean and Variance

- For each mini-batch, calculate:
 - mean
 - variance
- This tells how the data is distributed

2. Normalize

- Adjust values to have:
 - mean ≈ 0
 - variance ≈ 1

(shift)



Batch Normalization (Step-by-Step)

Batch Normalization works on the activations of a layer for a mini-batch of data to keep values stable and improve learning.

1) Compute Mean and Variance

For each neuron, we first look at its outputs over a mini-batch of size M .

Mean: This is the average activation value in the whole mini-batch.

Variance / Spread: This measures how much the values vary around the mean.

2) Normalize

Then it normalizes the activations. This means:

- subtract the mean
- divide by the standard deviation

So the values become more stable and easier for the network to learn from.

3) Scale and Shift

After normalization, the network applies two learnable parameters

- γ = scale
- β = shift

This is important because the network can still adjust the normalized values if needed. So BatchNorm does not force the data to stay fixed forever.

4) Moving Average

During training, mean and standard deviation are calculated for each mini-batch. But during testing, the model may see new data one sample at a time, so it needs stable values.

That is why it keeps a moving average:

$$\mu_{\text{mov}} = \alpha \mu_{\text{mov}} + (1 - \alpha) \mu_i$$

$$\sigma_{\text{mov}} = \alpha \sigma_{\text{mov}} + (1 - \alpha) \sigma_i$$

These running values are used during testing/inference.

📌 **Given:**

Assume a neuron outputs these activations for a mini-batch of **M = 4 samples**:

$$A = [2, 4, 6, 8]$$

We will apply Batch Normalization to this.

1 Step 1: Compute Mean

$$\mu = \frac{2 + 4 + 6 + 8}{4} = \frac{20}{4} = 5$$

2 Step 2: Compute Variance and Standard Deviation

Variance:

$$\begin{aligned} \sigma^2 &= \frac{(2 - 5)^2 + (4 - 5)^2 + (6 - 5)^2 + (8 - 5)^2}{4} \\ &= \frac{9 + 1 + 1 + 9}{4} = \frac{20}{4} = 5 \end{aligned}$$

Standard deviation:

$$\sigma = \sqrt{5} \approx 2.236$$

3 Step 3: Normalize

$$\hat{A}_i = \frac{A_i - \mu}{\sigma}$$

Now apply to each value:

For 2:

$$\frac{2 - 5}{2.236} = \frac{-3}{2.236} \approx -1.34$$

For 4:

$$\frac{4 - 5}{2.236} \approx -0.45$$

For 6:

$$\frac{6 - 5}{2.236} \approx 0.45$$

For 8:

$$\frac{8 - 5}{2.236} \approx 1.34$$

So normalized output:

$$\hat{A} = [-1.34, -0.45, 0.45, 1.34]$$

4 Step 4: Scale and Shift

Now apply learnable parameters:

Let:

- $\gamma = 2$
- $\beta = 1$

Formula:

$$y = \gamma \hat{A} + \beta$$

Apply to each value:

- $2(-1.34) + 1 = -2.68 + 1 = -1.68$
- $2(-0.45) + 1 = -0.90 + 1 = 0.10$
- $2(0.45) + 1 = 0.90 + 1 = 1.90$
- $2(1.34) + 1 = 2.68 + 1 = 3.68$

🎯 Final Output After BatchNorm:

$$[-1.68, 0.10, 1.90, 3.68]$$